

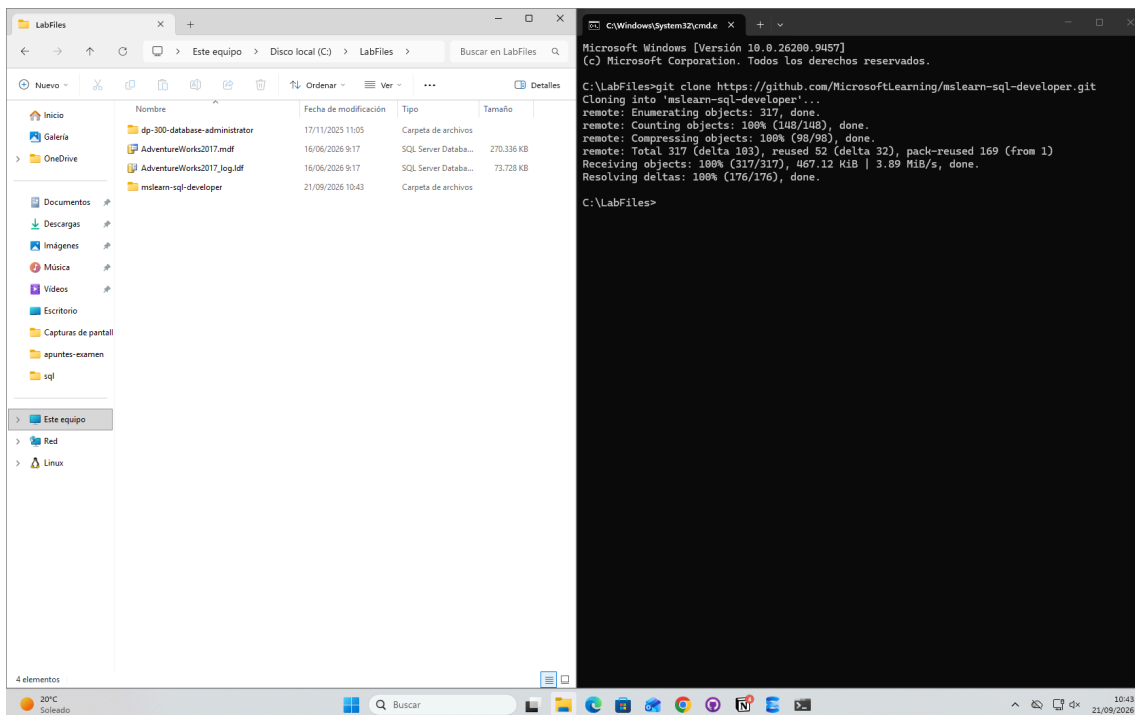
# Práctica 1: Diseño e implementación de objetos de base de datos con SQL

## 1. Configuración del entorno

Cloné el repositorio del laboratorio desde GitHub en Visual Studio Code (requiere SQL Server 2025+).

Usé la opción `Git: Clone` con la URL:

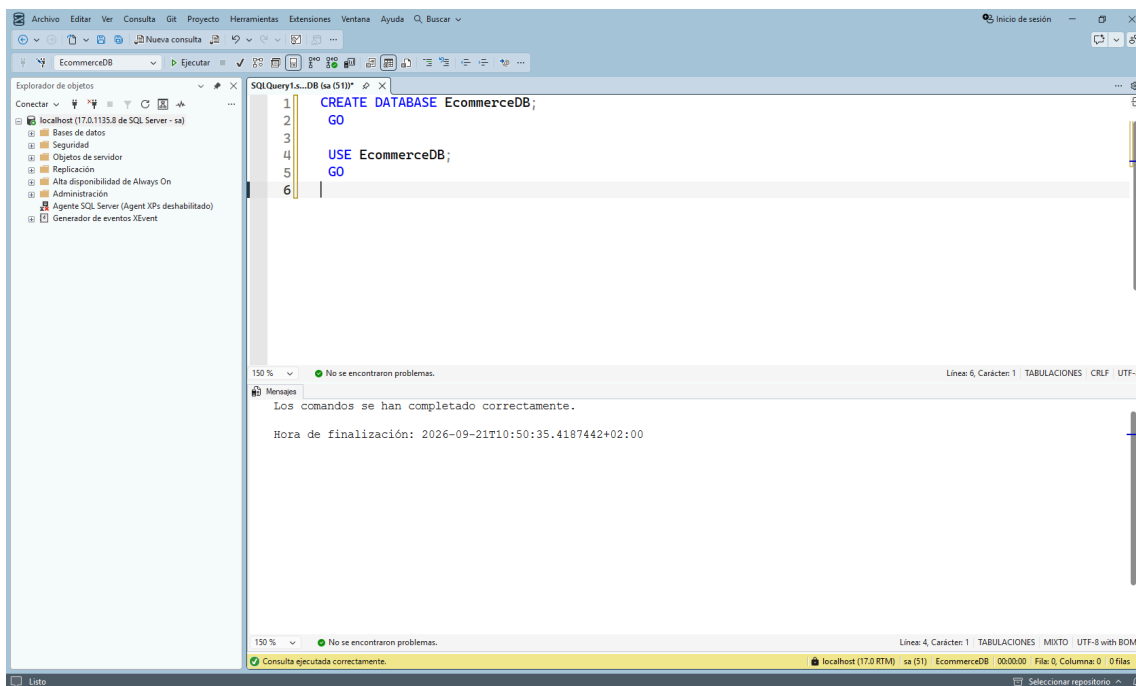
```
https://github.com/MicrosoftLearning/mslearn-sql-developer.git
```



## 2. Creación de la base de datos

En SQL Server Management Studio (SSMS), ejecuté el siguiente script para crear la base de datos `EcommerceDB` :

```
CREATE DATABASE EcommerceDB;  
  
GO  
  
USE EcommerceDB;  
  
GO
```



### 3. Tablas principales y restricciones

Creé las tablas de proveedores ( `Supplier` ), categorías ( `Category` ) y productos ( `Product` ), configurando claves y restricciones `CHECK` para garantizar la integridad de los datos.

```
USE EcommerceDB;
GO

-- Create Supplier table
CREATE TABLE Supplier (
    SupplierID INT PRIMARY KEY IDENTITY(1,1),
    SupplierName NVARCHAR(100) NOT NULL UNIQUE,
    Country NVARCHAR(50) NOT NULL,
    Email NVARCHAR(100),
    Phone NVARCHAR(20),
    CreatedDate DATETIME2 DEFAULT GETUTCDATE()
);

-- Create Category table
CREATE TABLE Category (
    CategoryID INT PRIMARY KEY IDENTITY(1,1),
    CategoryName NVARCHAR(100) NOT NULL UNIQUE,
    Description NVARCHAR(500)
);

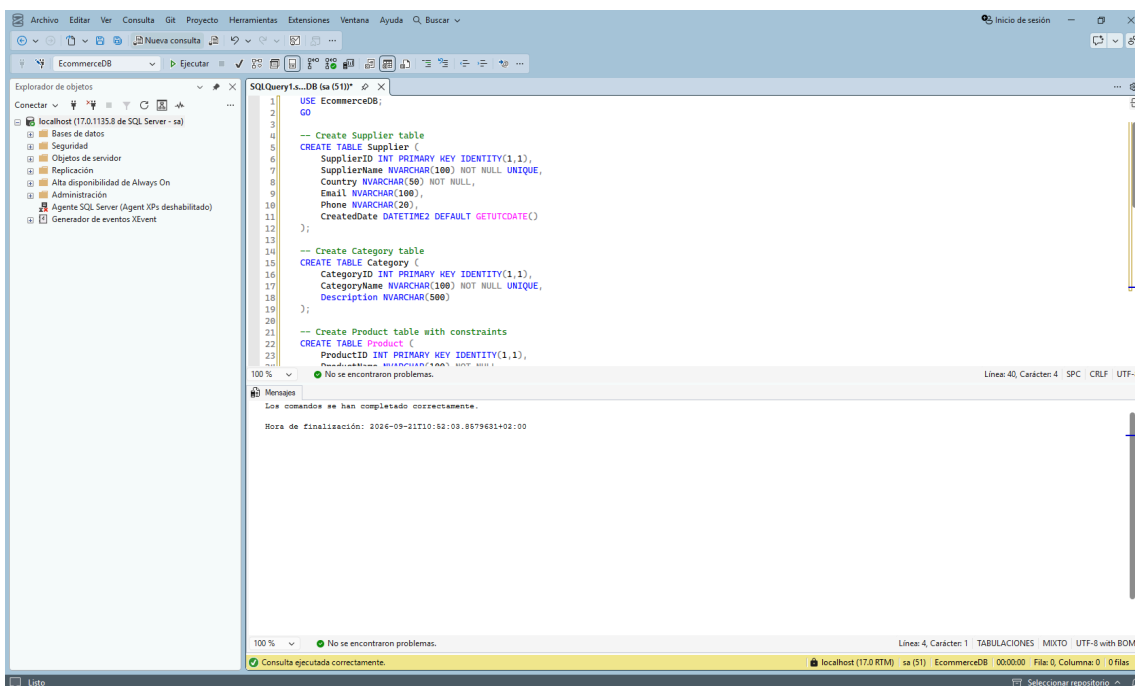
-- Create Product table with constraints
CREATE TABLE Product (
    ProductID INT PRIMARY KEY IDENTITY(1,1),
    ProductName NVARCHAR(100) NOT NULL,
```

```

    CategoryID INT NOT NULL,
    SupplierID INT NOT NULL,
    BasePrice DECIMAL(10,2) NOT NULL,
    StockQuantity INT NOT NULL DEFAULT 0,
    CreatedDate DATETIME2 DEFAULT GETUTCDATE(),
    CHECK (BasePrice > 0),
    CHECK (StockQuantity >= 0),
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID),
    FOREIGN KEY (SupplierID) REFERENCES Supplier(SupplierID)
);

-- Create indexes
CREATE INDEX IX_Category ON Product(CategoryID);
CREATE INDEX IX_Supplier ON Product(SupplierID);
GO

```



Luego inserté datos de prueba para verificar el esquema.

```

USE EcommerceDB;
GO

-- Insert sample suppliers
INSERT INTO Supplier (SupplierName, Country, Email, Phone)
VALUES
    ('Contoso Supplies', 'USA', 'contact@contoso.com', '555-0100'),
    ('Fabrikam Inc', 'Canada', 'sales@fabrikam.com', '555-0200');

-- Insert sample categories
INSERT INTO Category (CategoryName, Description)

```

VALUES

```
('Electronics', 'Electronic devices and accessories'),  
( 'Clothing', 'Apparel and fashion items');
```

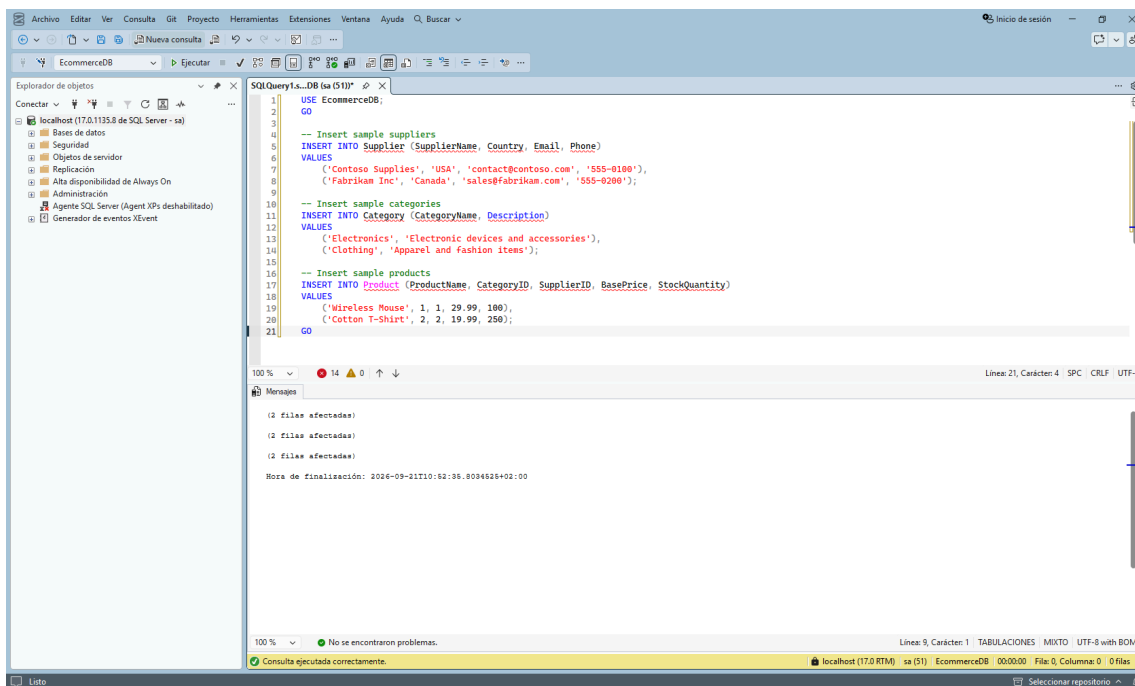
-- Insert sample products

```
INSERT INTO Product (ProductName, CategoryID, SupplierID, BasePrice, StockQuantity)
```

VALUES

```
('Wireless Mouse', 1, 1, 29.99, 100),  
( 'Cotton T-Shirt', 2, 2, 19.99, 250);
```

GO



## 4. Tabla temporal (Historial de precios)

Para registrar el historial de precios, creé la tabla `ProductPrice` con control de versiones ( `SYSTEM_VERSIONING` ).

```
USE EcommerceDB;
```

GO

-- Create Price History table with temporal versioning

```
CREATE TABLE ProductPrice (  
    PriceID INT PRIMARY KEY IDENTITY(1,1),  
    ProductID INT NOT NULL,  
    CurrentPrice DECIMAL(10,2) NOT NULL,  
    EffectiveDate DATE,  
    SysStartTime DATETIME2 GENERATED ALWAYS AS ROW START HIDDEN,  
    SysEndTime DATETIME2 GENERATED ALWAYS AS ROW END HIDDEN,  
    PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime),  
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)  
) WITH (SYSTEM_VERSIONING = ON);
```

GO

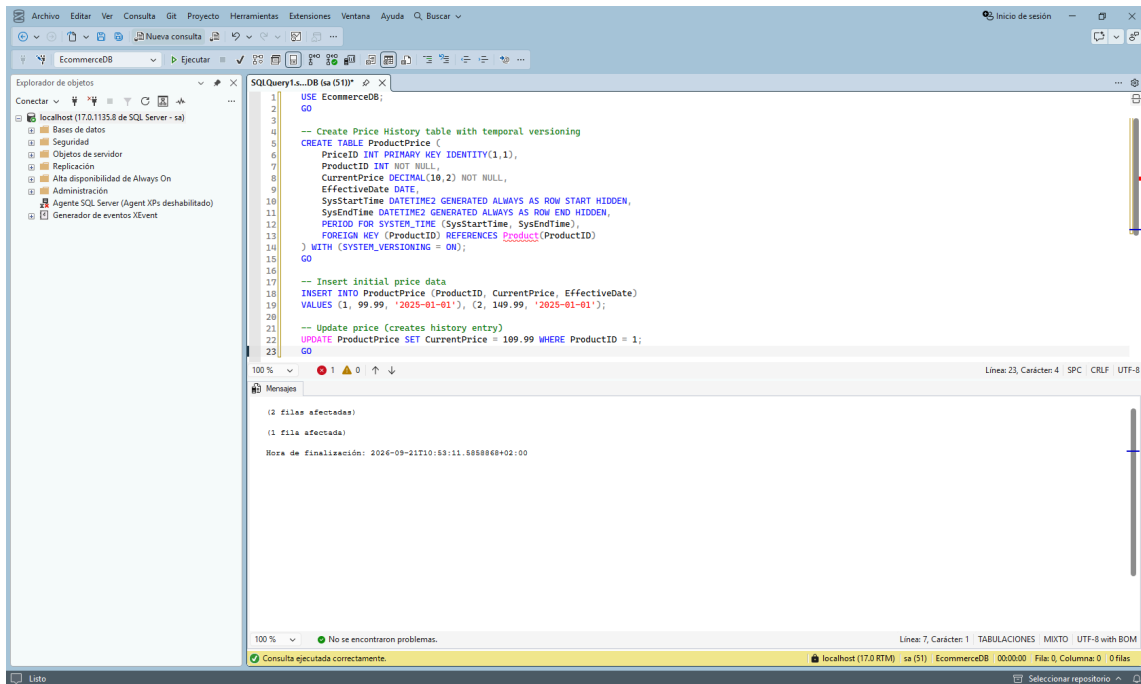
-- Insert initial price data

```
INSERT INTO ProductPrice (ProductID, CurrentPrice, EffectiveDate)
VALUES (1, 99.99, '2025-01-01'), (2, 149.99, '2025-01-01');
```

-- Update price (creates history entry)

```
UPDATE ProductPrice SET CurrentPrice = 109.99 WHERE ProductID = 1;
```

GO



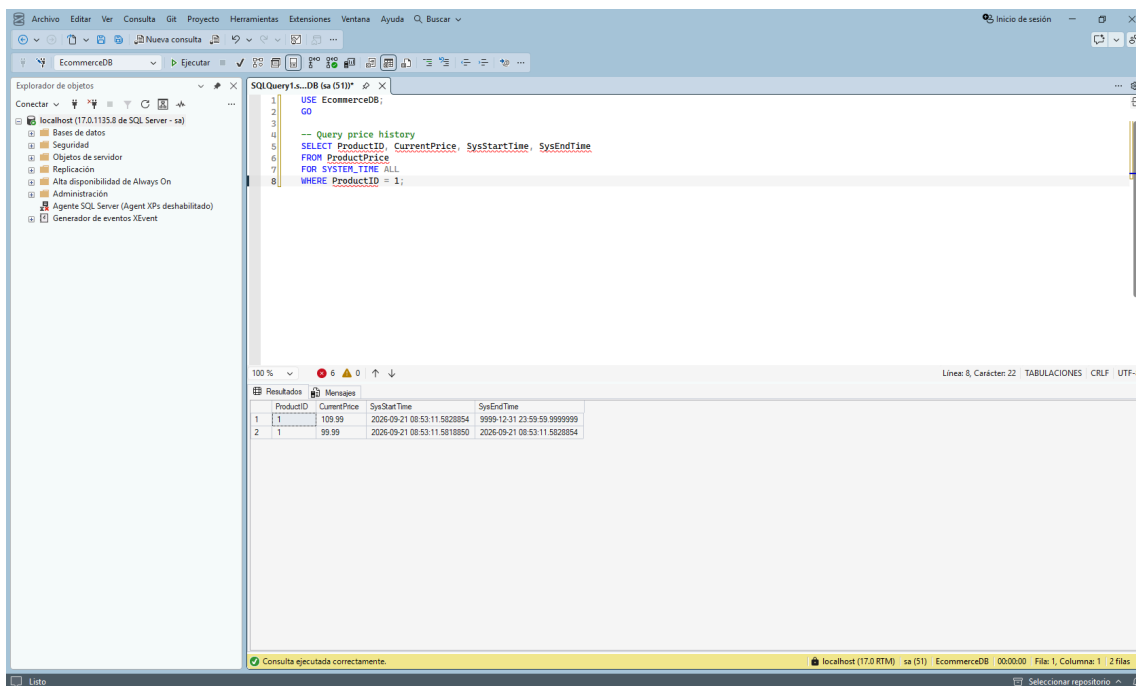
Consultando con `FOR SYSTEM_TIME ALL` , el sistema muestra el precio actual y los anteriores con sus fechas.

```
USE EcommerceDB;
```

GO

-- Query price history

```
SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime
FROM ProductPrice
FOR SYSTEM_TIME ALL
WHERE ProductID = 1;
```



## 5. Columnas JSON para metadatos

Añadí una columna JSON a la tabla `Product` para almacenar propiedades variables, junto con una columna calculada para el "color" y un índice para agilizar búsquedas.

```
USE EcommerceDB;
GO

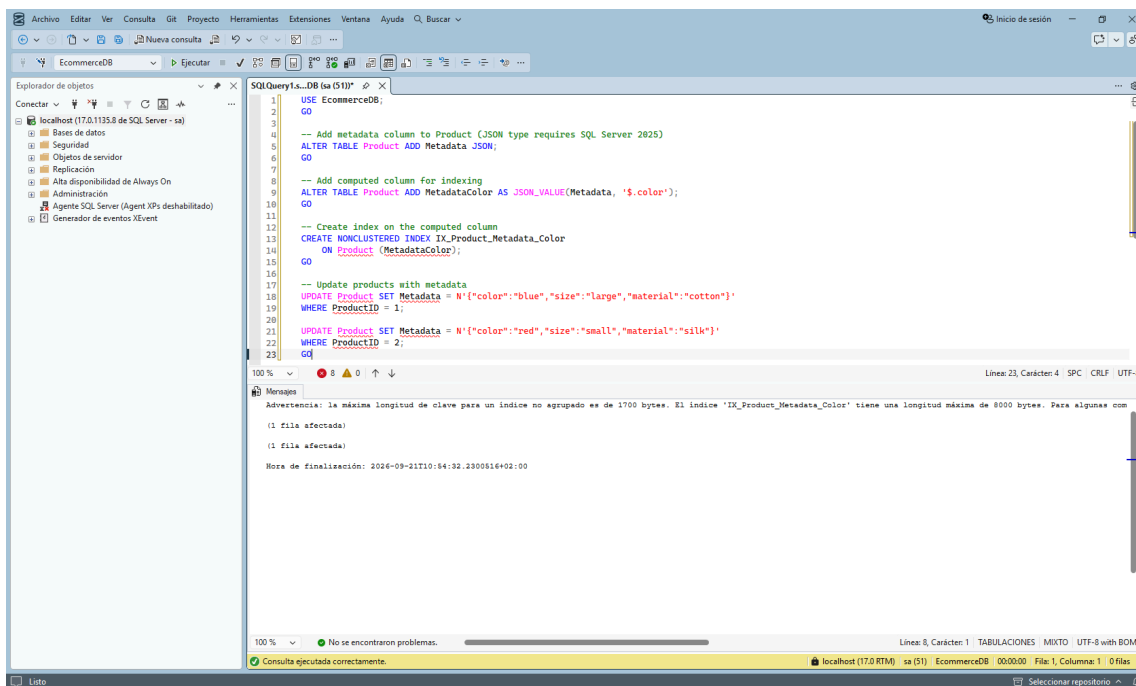
-- Add metadata column to Product (JSON type requires SQL Server 2025)
ALTER TABLE Product ADD Metadata JSON;
GO

-- Add computed column for indexing
ALTER TABLE Product ADD MetadataColor AS JSON_VALUE(Metadata, '$.color');
GO

-- Create index on the computed column
CREATE NONCLUSTERED INDEX IX_Product_Metadata_Color
ON Product (MetadataColor);
GO

-- Update products with metadata
UPDATE Product SET Metadata = N'{"color":"blue","size":"large","material":"cotton"}'
WHERE ProductID = 1;

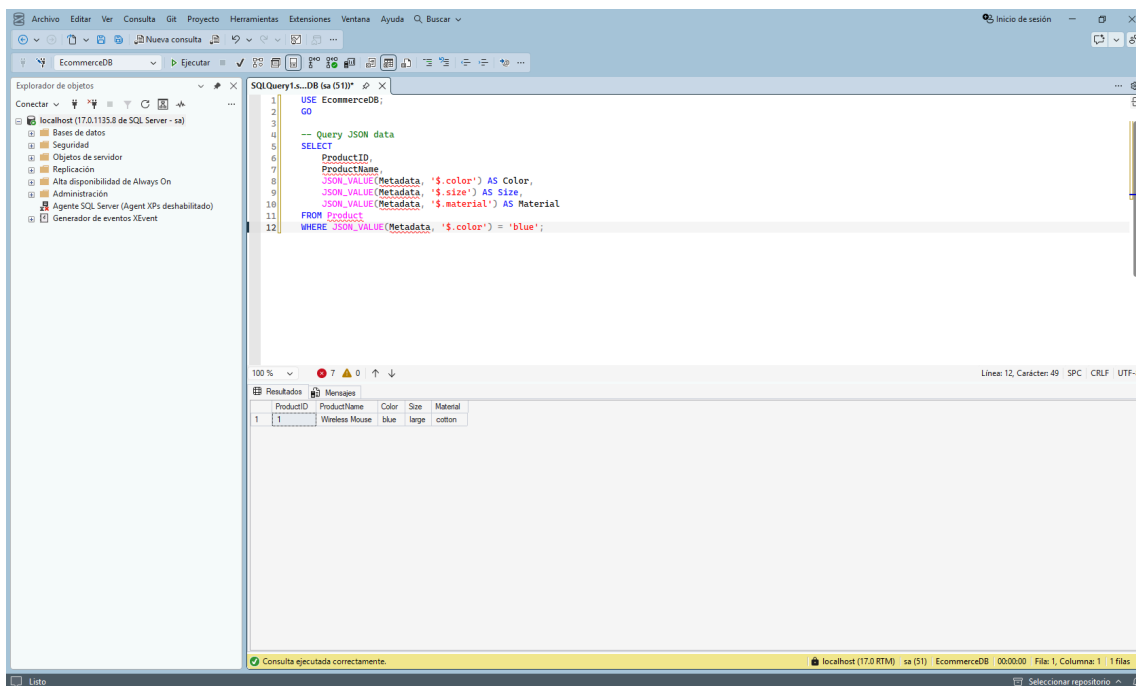
UPDATE Product SET Metadata = N'{"color":"red","size":"small","material":"silk"}'
WHERE ProductID = 2;
GO
```



Luego, extraje propiedades específicas usando `JSON_VALUE` .

```
USE EcommerceDB;
GO

-- Query JSON data
SELECT
    ProductID,
    ProductName,
    JSON_VALUE(Metadata, '$.color') AS Color,
    JSON_VALUE(Metadata, '$.size') AS Size,
    JSON_VALUE(Metadata, '$.material') AS Material
FROM Product
WHERE JSON_VALUE(Metadata, '$.color') = 'blue';
```



## 6. Tabla de pedidos particionada

Para mejorar el rendimiento al manejar grandes volúmenes de datos, particioné la tabla `Order` por trimestres.

```
USE EcommerceDB;
GO

-- Create partition function for order dates
-- Use RANGE RIGHT for date columns to keep same-day values together
CREATE PARTITION FUNCTION PF_OrderDate (DATE)
    AS RANGE RIGHT FOR VALUES
    ('2025-01-01', '2025-04-01', '2025-07-01', '2025-10-01');

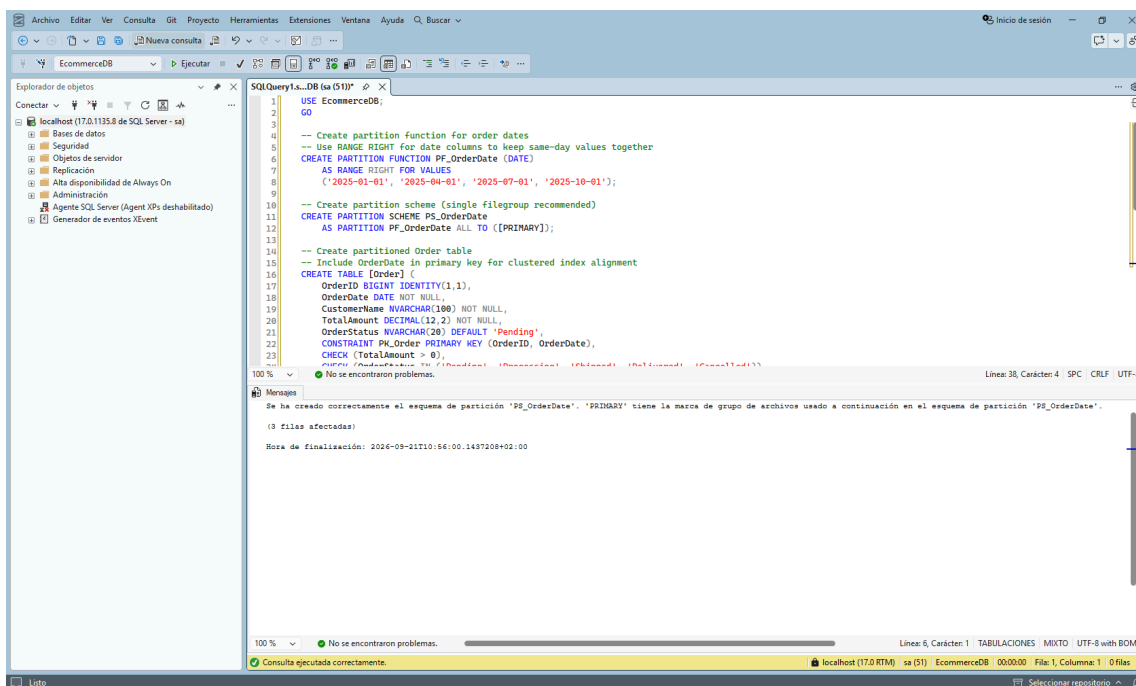
-- Create partition scheme (single filegroup recommended)
CREATE PARTITION SCHEME PS_OrderDate
    AS PARTITION PF_OrderDate ALL TO ([PRIMARY]);

-- Create partitioned Order table
-- Include OrderDate in primary key for clustered index alignment
CREATE TABLE [Order] (
    OrderID BIGINT IDENTITY(1,1),
    OrderDate DATE NOT NULL,
    CustomerName NVARCHAR(100) NOT NULL,
    TotalAmount DECIMAL(12,2) NOT NULL,
    OrderStatus NVARCHAR(20) DEFAULT 'Pending',
    CONSTRAINT PK_Order PRIMARY KEY (OrderID, OrderDate),
    CHECK (TotalAmount > 0),
    CHECK (OrderStatus IN ('Pending', 'Processing', 'Shipped', 'Delivered', 'Cancelled'))
) ON PS_OrderDate(OrderDate);
```



```
-- Create partitioned index
CREATE NONCLUSTERED INDEX IX_Order_Customer
    ON [Order](CustomerName)
    ON PS_OrderDate(OrderDate);
GO

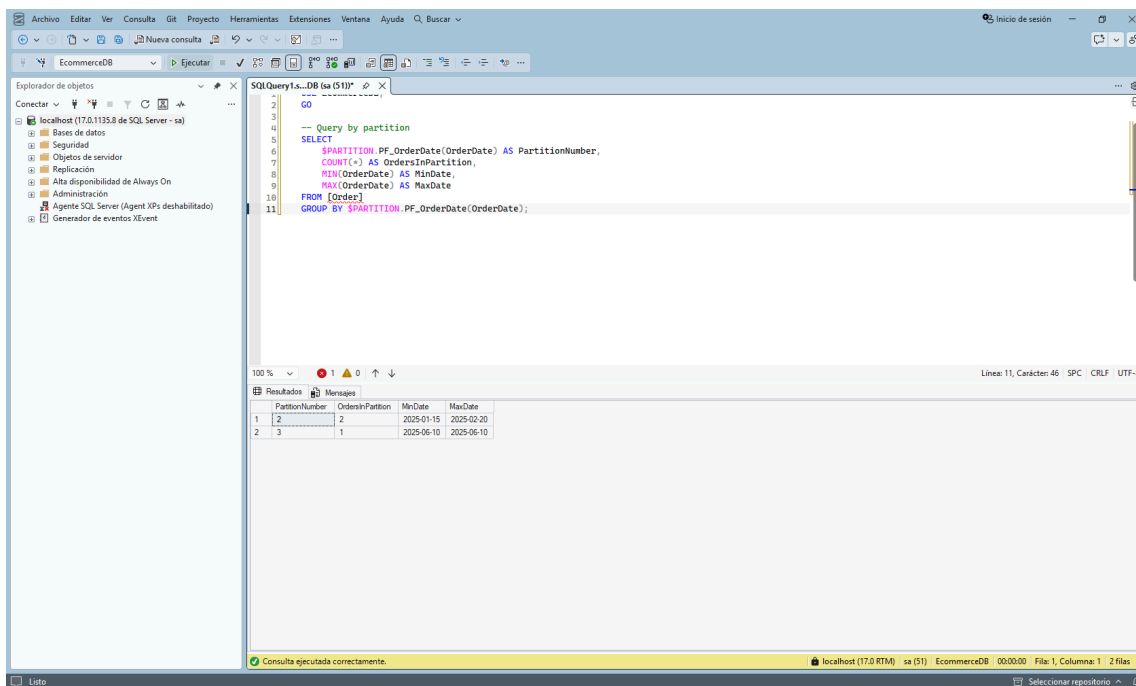
-- Insert sample orders
INSERT INTO [Order] (OrderDate, CustomerName, TotalAmount, OrderStatus) VALUES
    ('2025-01-15', 'John Smith', 299.97, 'Delivered'),
    ('2025-02-20', 'Jane Doe', 149.99, 'Shipped'),
    ('2025-06-10', 'Bob Johnson', 449.95, 'Processing');
GO
```



Comprobé con `$PARTITION` que los registros se distribuyen correctamente por fecha.

```
USE EcommerceDB;
GO

-- Query by partition
SELECT
    $PARTITION.PF_OrderDate(OrderDate) AS PartitionNumber,
    COUNT(*) AS OrdersInPartition,
    MIN(OrderDate) AS MinDate,
    MAX(OrderDate) AS MaxDate
FROM [Order]
GROUP BY $PARTITION.PF_OrderDate(OrderDate);
```



## 7. Identificadores con SEQUENCE

Usé un objeto `SEQUENCE` independiente para generar los IDs de las líneas de pedido ( `OrderDetail` ).

```

USE EcommerceDB;
GO

-- Create SEQUENCE for order line items
CREATE SEQUENCE OrderLineSequence
    START WITH 1
    INCREMENT BY 1;

-- Create OrderDetail table
CREATE TABLE OrderDetail (
    OrderLineID INT PRIMARY KEY,
    OrderID BIGINT NOT NULL,
    OrderDate DATE NOT NULL,
    ProductID INT NOT NULL,
    Quantity INT NOT NULL,
    UnitPrice DECIMAL(10,2) NOT NULL,
    LineTotal AS (Quantity * UnitPrice),
    CHECK (Quantity > 0),
    CHECK (UnitPrice > 0),
    FOREIGN KEY (OrderID, OrderDate) REFERENCES [Order](OrderID, OrderDate),
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);
GO

-- Insert order details using SEQUENCE

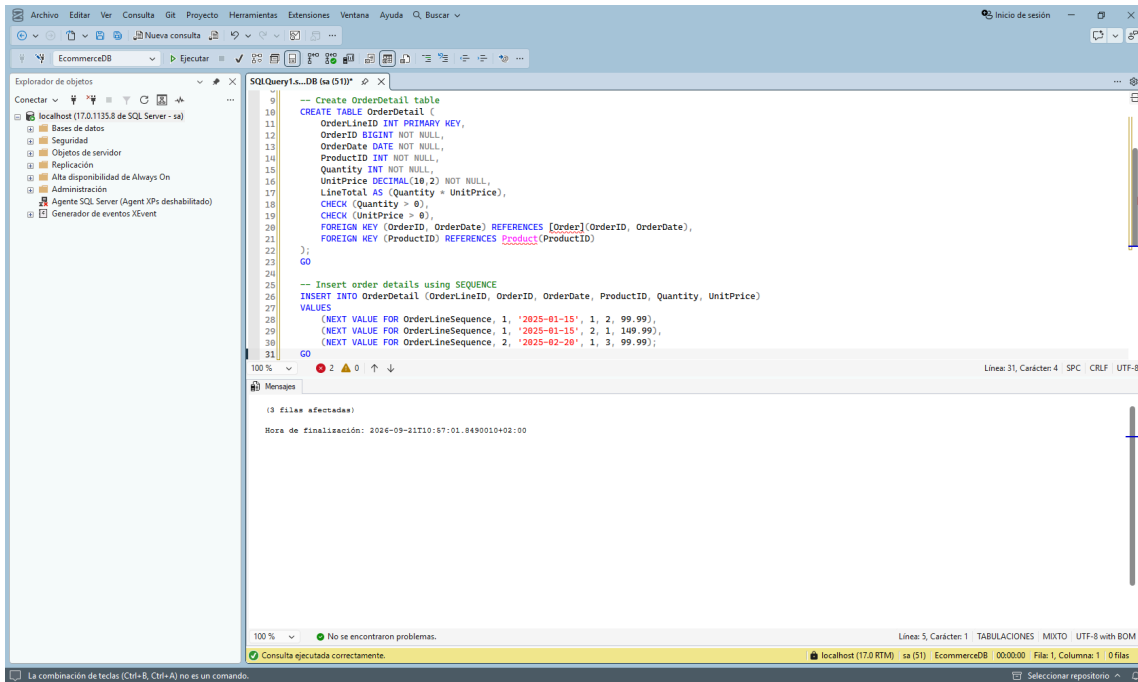
```

```

INSERT INTO OrderDetail (OrderLineID, OrderID, OrderDate, ProductID, Quantity, UnitPrice)
VALUES
    (NEXT VALUE FOR OrderLineSequence, 1, '2025-01-15', 1, 2, 99.99),
    (NEXT VALUE FOR OrderLineSequence, 1, '2025-01-15', 2, 1, 149.99),
    (NEXT VALUE FOR OrderLineSequence, 2, '2025-02-20', 1, 3, 99.99);

GO

```



Verifiqué que los números se generaron de forma correlativa.

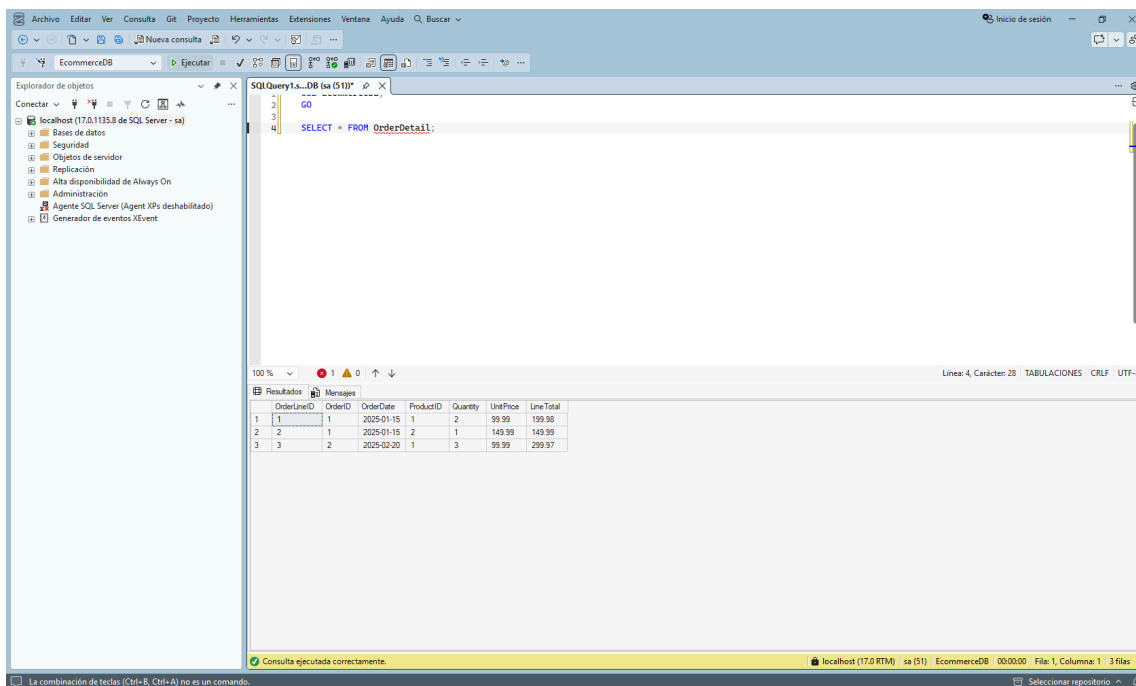
```

USE EcommerceDB;

GO

SELECT * FROM OrderDetail;

```

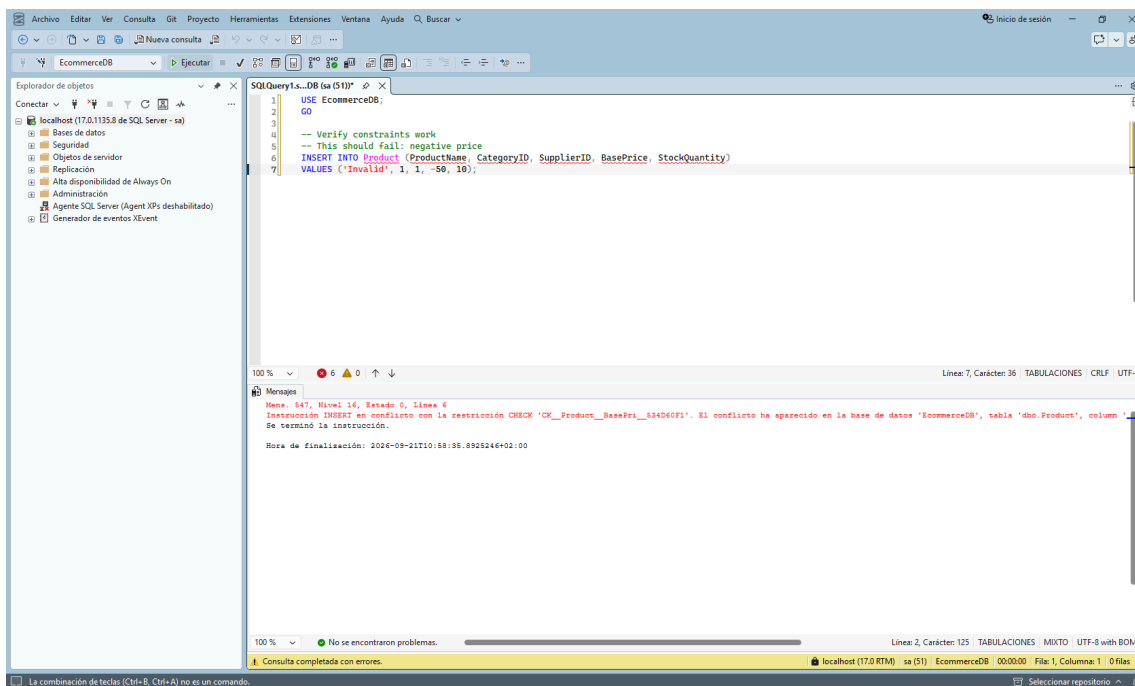


## 8. Verificación final

Intenté insertar un precio negativo para comprobar que la restricción `CHECK` bloquea el error correctamente.

```
USE EcommerceDB;
GO

-- Verify constraints work
-- This should fail: negative price
INSERT INTO Product (ProductName, CategoryID, SupplierID, BasePrice, StockQuantity)
VALUES ('Invalid', 1, 1, -50, 10);
```



Por último, realicé consultas finales de validación sobre JSON, particiones e historial temporal.

```
USE EcommerceDB;
GO

-- Verify JSON queries work
SELECT ProductName, JSON_VALUE(Metadata, '$.color') AS Color
FROM Product
WHERE Metadata IS NOT NULL;

-- Verify partitioning
SELECT $PARTITION.PF_OrderDate(OrderDate) AS Partition, COUNT(*) AS RecordCount
FROM [Order]
GROUP BY $PARTITION.PF_OrderDate(OrderDate);

-- Verify temporal table
SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime
FROM ProductPrice FOR SYSTEM_TIME ALL
ORDER BY ProductID, SysStartTime;
```

ArchivoEditarVerConsultaGitProyectoHerramientasExtensionesVentanaAyudaBuscar

Inicio de sesión

EcommerceDBEjecutar

Explorador de objetos

Conectarlocalhost (172.1135.8 de SQL Server - sa)

- Bases de datos
- Seguridad
- Objetos de servidor
- Replicación
- Alta disponibilidad de Always On
- Administración
- Agente SQL Server (Agent XPs deshabilitado)
- Generador de eventos XEvent

SQLQuery1.s...DB (sa (51))

```
1 USE EcommerceDB;
2 GO
3
4 -- Verify JSON queries work
5 SELECT ProductName, JSON_VALUE(Metadata, '$.color') AS Color
6 FROM Product
7 WHERE Metadata IS NOT NULL;
8
9 -- Verify partitioning
10 SELECT $PARTITION.PF_OrderDate(OrderDate) AS Partition, COUNT(*) AS RecordCount
11 FROM [Order]
12 GROUP BY $PARTITION.PF_OrderDate(OrderDate);
13
14 -- Verify temporal table
15 SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime
16 FROM ProductPrice FOR SYSTEM_TIME ALL
17 ORDER BY ProductID, SysStartTime;
```

100 % 12 0

Linea: 17, Carácter: 35 TABULACIONES CRLF UTF-8

Resultados Mensajes

|   | ProductName    | Color |
|---|----------------|-------|
| 1 | Wireless Mouse | blue  |
| 2 | Cotton T-Shirt | red   |

|   | Partition | RecordCount |
|---|-----------|-------------|
| 1 | 2         | 2           |
| 2 | 3         | 1           |

|   | ProductID | CurrentPrice | SysStartTime                | SysEndTime                  |
|---|-----------|--------------|-----------------------------|-----------------------------|
| 1 | 1         | 99.99        | 2026-09-21 08:53:11.5818850 | 2026-09-21 08:53:11.5828854 |
| 2 | 1         | 109.99       | 2026-09-21 08:53:11.5828854 | 9999-12-31 23:59:59.9999999 |
| 3 | 2         | 149.99       | 2026-09-21 08:53:11.5818850 | 9999-12-31 23:59:59.9999999 |

Consulta ejecutada correctamente.

localhost (172.1135.8 de SQL Server) sa (51) EcommerceDB 00:00:00 Fila: 1, Columna: 1 7 Filas

La combinación de teclas (Ctrl+B, Ctrl+A) no es un comando.

Seleccionar repositorio